CENTRO DE ENSINO SUPERIOR DE MARINGÁ –
UNICESUMAR

CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: ENGENHARIA DE SOFTWARE

YURI ORUPITIUK DA ROCHA
KELVIN HIDEKI SANADA

CRIAÇÃO E DESTRUIÇÃO DE OBJETOS E MENSAGENS SÍNCRONAS E ASSÍNCRONAS EM UMA LOJA VIRTUAL

TEMA: DIAGRAMAS DE SEQUÊNCIA

CIDADE
2026

SUMÁRIO

1 INTRODUÇÃO 3
1.1 Tema e objetivo 3
1.2 Cenário escolhido 3
1.3 Organização do trabalho 3
2 FUNDAMENTAÇÃO 3
2.1 O diagrama de sequência 3
2.2 Mensagens síncronas e assíncronas 4
2.3 Barra de ativação 4
2.4 Criação e destruição de objetos 4
3 MODELAGEM DO CENÁRIO 5
3.1 Diagrama 1 – Finalização da compra 5
3.2 Diagrama 2 – Pagamento assíncrono e confirmação 6
4 ANÁLISE 7
5 CONCLUSÃO 8
REFERÊNCIAS 9

1 INTRODUÇÃO

1.1 Tema e objetivo

Este trabalho trata do tema 3 da disciplina de Engenharia de Software: o diagrama de sequência da UML (*Unified Modeling Language*), com foco na criação e na destruição de objetos e nas mensagens síncronas e assíncronas. Também são abordados a mensagem de retorno, a autochamada e a barra de ativação, elementos que dão sentido temporal ao diagrama.

O objetivo é estudar esses elementos, explicar a diferença entre os tipos de mensagem e de seta e aplicá-los na modelagem de um cenário de software, usando uma ferramenta CASE. Os diagramas produzidos usam criação de objetos (*create*), destruição de objetos e mensagens assíncronas, e cada um é acompanhado de uma leitura passo a passo.

1.2 Cenário escolhido

O cenário é uma loja virtual de eletrônicos, no processo de finalização da compra (*checkout*). O cliente monta um carrinho e confirma a compra. O sistema confere os itens, reserva o estoque, cria o pedido e calcula o valor total. A partir daí o carrinho deixa de ter utilidade e é descartado, pois seus itens já passaram para o pedido.

Em seguida o pagamento é cobrado por um gateway externo, que pode demorar a responder e avisa o resultado por notificação. Quando o pagamento é aprovado, o pedido é confirmado, um comprovante é gerado, a reserva temporária de estoque é encerrada e um e-mail de confirmação é enviado sem que o sistema precise esperar por ele. O cenário foi escolhido porque reúne, de forma natural, objetos que nascem (pedido, comprovante), objetos que morrem (carrinho, reserva) e sistemas externos que respondem de modo assíncrono.

1.3 Organização do trabalho

A seção 2 apresenta a fundamentação com os conceitos do tema. A seção 3 traz os dois diagramas do cenário, cada um seguido de explicação. A seção 4 analisa as decisões de modelagem e as dificuldades encontradas, e a seção 5 apresenta a conclusão.

2 FUNDAMENTAÇÃO

2.1 O diagrama de sequência

O diagrama de sequência é um diagrama de interação da UML que mostra como objetos trocam mensagens ao longo do tempo para realizar um comportamento,

normalmente o fluxo de um caso de uso (FOWLER, 2005; BOOCH; RUMBAUGH; JACOBSON, 2006). Cada participante é representado por uma *linha de vida*: um retângulo com o nome do objeto, de onde desce uma linha tracejada vertical. O tempo corre de cima para baixo, de modo que uma mensagem desenhada mais abaixo ocorre depois de uma desenhada mais acima.

2.2 Mensagens síncronas e assíncronas

Uma mensagem **síncrona** indica que quem envia fica bloqueado até receber a resposta, como em uma chamada comum de método. Na notação da UML ela é desenhada como uma linha cheia com ponta de seta preenchida. Uma mensagem **assíncrona** indica que o remetente continua seu trabalho sem esperar a resposta, como no envio de uma notificação ou no disparo de uma requisição a um serviço lento. Ela é desenhada como uma linha cheia com ponta de seta aberta, em forma de “V” (OMG, 2017; FOWLER, 2005).

A **mensagem de retorno** é desenhada como linha tracejada com ponta aberta e leva de volta ao chamador o resultado ou a confirmação de uma chamada síncrona. Mensagens assíncronas não têm retorno implícito: se houver resposta, ela é uma nova mensagem assíncrona, enviada no sentido contrário. A **autochamada** é uma mensagem que sai de uma linha de vida e volta para ela mesma, usada quando o objeto executa uma operação própria, como validar dados que ele mesmo mantém.

2.3 Barra de ativação

A **barra de ativação**, também chamada de especificação de execução, é um retângulo estreito sobre a linha de vida que mostra o intervalo em que o objeto está executando uma operação. Ela começa quando a mensagem é recebida e termina quando a operação acaba, em geral no retorno. Quando uma operação chama outra no mesmo objeto, desenha-se uma barra menor sobreposta à primeira. Na mensagem assíncrona, a barra do remetente termina logo após o envio, pois ele não espera, e o destinatário abre uma nova barra quando começa a tratar a mensagem.

2.4 Criação e destruição de objetos

Nem todo objeto existe desde o início da interação. A **criação** é representada por uma mensagem que termina na caixa do próprio objeto, colocada na altura em que ele nasce, e não no topo do diagrama. A UML 2.5.1 usa para isso uma linha tracejada com ponta aberta, rotulada com o estereótipo «create» (OMG, 2017). A **destruição** é representada por um “X” no fim da linha de vida, logo depois da mensagem que encerra o objeto; nada pode ser enviado a ele depois desse ponto. Esses dois recursos expressam o ciclo de vida: o objeto só recebe mensagens entre o seu nascimento e a sua morte.

Tabela 1 – Elementos do diagrama de sequência e sua notação

Elemento	Notação	Significado e exemplo no cenário
Linha de vida	Retângulo com linha vertical tracejada	Participante da interação. Ex.: Carrinho, Estoque.
Barra de ativação	Retângulo estreito sobre a linha de vida	Período em que o objeto executa uma operação. Ex.: PedidoController durante finalizarCompra().
Mensagem síncrona	Linha cheia, seta preenchida	Remetente espera a resposta. Ex.: obterItens().
Mensagem assíncrona	Linha cheia, seta aberta	Remetente não espera. Ex.: solicitarCobranca().
Mensagem de retorno	Linha tracejada, seta aberta	Devolve o resultado. Ex.: listaDeItens.
Autochamada	Seta que volta à mesma linha de vida	Operação interna. Ex.: validarItens().
Criação	Seta tracejada até a caixa do objeto, «create»	Objeto nasce no meio do fluxo. Ex.: Pedido, Comprovante.
Destruição	“X” no fim da linha de vida	Objeto deixa de existir. Ex.: Carrinho, ReservaEstoque.

Fonte: elaborado pelos autores (2026).

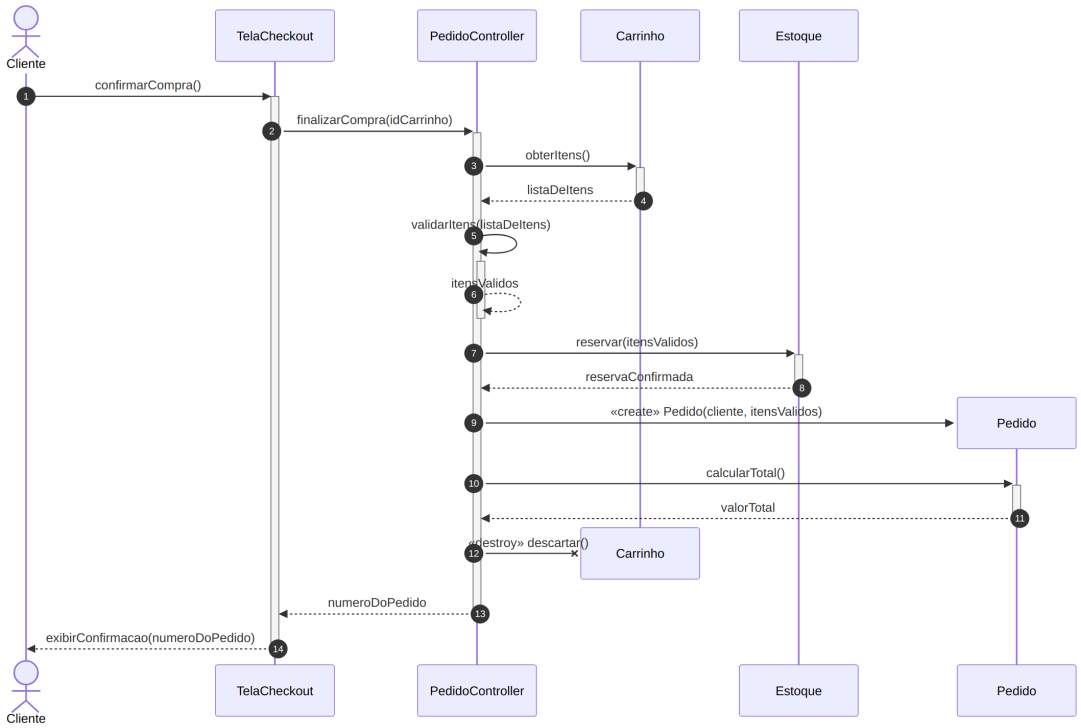
3 MODELAGEM DO CENÁRIO

Os diagramas foram escritos em Mermaid, ferramenta CASE baseada em texto, e exportados em PNG e SVG. Os arquivos-fonte (.mmd) estão na pasta diagramas do repositório. Os nomes das linhas de vida seguem o padrão Interface, Controle e Entidade: TelaCheckout é a interface, PedidoController e PagamentoController são os controles, e Carrinho, Estoque, Pedido, Comprovante e ReservaEstoque são entidades. GatewayPagamento e ServicoEmail são sistemas externos. As mensagens estão numeradas na ordem em que ocorrem.

3.1 Diagrama 1 – Finalização da compra

O Diagrama 1, na Figura 1, mostra o fluxo desde a confirmação da compra pelo cliente até a exibição do número do pedido. Ele usa mensagens síncronas, retornos, uma autochamada, a criação do Pedido e a destruição do Carrinho.

Figura 1 – Diagrama de sequência da finalização da compra



Fonte: elaborado pelos autores (2026).

A leitura é a seguinte. Na mensagem 1, o Cliente confirma a compra na TelaCheckout, que abre sua barra de ativação e, na mensagem 2, repassa a solicitação ao PedidoController. O controle pede os itens ao Carrinho (mensagem 3), que os devolve (4). Em seguida o controle executa a autochamada validarItens() (5), que confere preço e disponibilidade e devolve o resultado a si mesmo (6); a barra menor, sobreposta à do controle, mostra que a validação acontece dentro da operação principal.

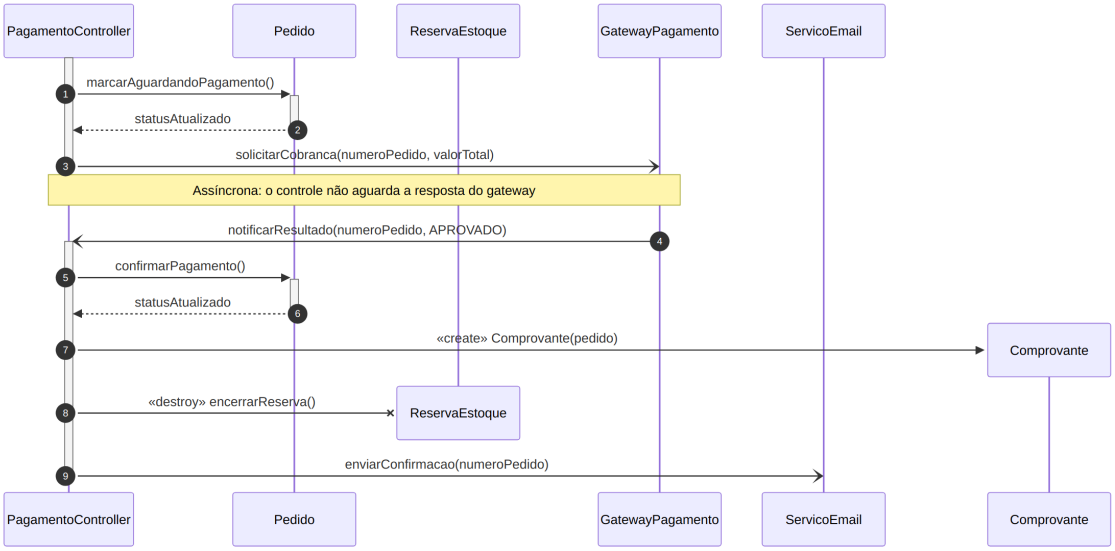
Com os itens válidos, o controle pede ao Estoque que reserve as unidades (7) e recebe a confirmação (8). Na mensagem 9 ocorre a **criação** do Pedido: a seta termina na caixa do objeto, desenhada na altura em que ele nasce, o que indica que antes desse ponto o pedido não existia. O controle então chama calcularTotal() (10) e recebe o valor (11). Na mensagem 12 ocorre a **destruição** do Carrinho, marcada pelo “X” no fim da sua linha de vida: seus itens já pertencem ao Pedido, e o carrinho não recebe mais mensagens. Por fim, o controle devolve o número do pedido à tela (13), que o exibe ao cliente (14).

3.2 Diagrama 2 – Pagamento assíncrono e confirmação

O Diagrama 2, na Figura 2, começa depois do Diagrama 1, quando o pedido já existe, e mostra a cobrança feita por um sistema externo. Aqui aparecem as mensagens assíncronas, uma nova criação (Comprovante) e uma nova destruição

(ReservaEstoque), objeto criado pelo Estoque na operação reservar() do Diagrama 1 e por isso representado já existente.

Figura 2 – Diagrama de sequência do pagamento assíncrono



Fonte: elaborado pelos autores (2026).

Nas mensagens 1 e 2, o PagamentoController muda o status do Pedido para aguardando pagamento, de forma síncrona. Na mensagem 3, solicitarCobranca() é enviada ao GatewayPagamento de forma **assíncrona**, com seta aberta: o controle não espera a resposta e, por isso, sua barra de ativação termina logo após o envio. A nota do diagrama registra essa decisão. O pagamento pode levar segundos ou minutos, e o sistema não fica travado nesse intervalo.

A resposta chega como uma nova mensagem assíncrona, notificarResultado() (4), enviada do gateway ao controle, que abre uma nova barra de ativação. O controle confirma o pagamento no Pedido (5 e 6), cria o Comprovante (7, criação) e encerra a ReservaEstoque (8, destruição), pois a baixa definitiva do estoque já foi registrada em confirmarPagamento(). Por último, enviarConfirmacao() (9) é uma mensagem assíncrona ao ServicoEmail, sem retorno, porque o envio do e-mail não deve atrasar nem impedir a conclusão da compra.

4 ANÁLISE

A primeira decisão foi escolher quais objetos nascem e quais morrem, com base no ciclo de vida de cada um. O Pedido só passa a existir quando a compra é confirmada, e o Comprovante só existe depois da aprovação do pagamento; por isso ambos são criados no meio do fluxo e não aparecem no topo. O Carrinho e a ReservaEstoque cumprem uma função temporária e perdem sentido quando essa função termina, o que justifica a destruição. O Cliente e o Estoque, ao contrário, continuam existindo, e por isso não receberam o “X”.

A segunda decisão foi usar mensagens assíncronas apenas onde há motivo técnico: a cobrança no gateway, que é lenta e externa, e o e-mail, cujo resultado não altera o fluxo principal. As chamadas entre objetos internos continuaram síncronas, com retorno. Como a resposta do gateway não vem pelo mesmo canal da requisição, ela foi modelada como uma segunda mensagem assíncrona em sentido contrário, e não como retorno.

As dificuldades foram principalmente de notação. Primeiro, decidir onde a barra de ativação do controle termina quando a mensagem é assíncrona. Segundo, manter a coerência entre os diagramas, já que a ReservaEstoque aparece como objeto existente no Diagrama 2 e foi criada fora dele. Terceiro, limitações da ferramenta: o Mermaid desenha as mensagens de criação com linha cheia e seta preenchida, e não tracejada e aberta como na UML 2.5.1, e desloca a caixa do participante destruído até o ponto do “X”. Por isso a criação é identificada pelo estereótipo «create» no rótulo e pela caixa na altura do nascimento. Também ficou de fora o caminho de falha do pagamento (recusa ou tempo esgotado), que exigiria um fragmento *alt* e pertence a outro tema.

5 CONCLUSÃO

O trabalho teve como objetivo estudar a criação e a destruição de objetos e as mensagens síncronas e assíncronas no diagrama de sequência. Ao modelar a finalização de uma compra em uma loja virtual, ficou claro que esses elementos expressam decisões reais de projeto: quando um objeto passa a existir, até quando ele é necessário e se o remetente pode ou não esperar uma resposta. A barra de ativação mostrou-se útil para enxergar quem está ocupado em cada momento, e a autochamada, para representar a lógica interna de um controle sem criar objetos desnecessários.

Aprendemos que o tipo de seta carrega significado e que trocá-la muda a leitura do diagrama, e que a resposta de um sistema externo assíncrono é uma nova mensagem, e não um retorno. Os pontos de maior dificuldade foram o posicionamento das barras de ativação nos trechos assíncronos, a coerência entre diagramas do mesmo cenário e as limitações da ferramenta na notação de criação e destruição. Como continuação, o cenário pode ser ampliado com o tratamento de falha do pagamento, usando fragmentos de decisão.

REFERÊNCIAS

BOOCH, Grady; RUMBAUGH, James; JACOBSON, Ivar. **UML**: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2006.

FOWLER, Martin. **UML essencial**: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

OBJECT MANAGEMENT GROUP (OMG). **Unified Modeling Language (UML)**: version 2.5.1. [S. l.]: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1. Acesso em: [preencher a data do seu acesso].